

Communication-Matched Warp Collectives for ML-KEM and ML-DSA on a RISC-V SIMT GPU

Anonymous Authors

Abstract—ML-KEM and ML-DSA share Keccak and number-theoretic transforms (NTTs), yet their useful parallelism has incompatible communication: Keccak repeatedly reduces and permutes a 25-word state, while an NTT exchanges coefficient pairs at power-of-two distances. We present communication-matched warp collectives that keep both states in ordinary lane registers of an RV32 RISC-V GPU. The instructions are stateless, use standard source vectors and single-destination writeback, and implement fixed Keccak neighborhoods or paired NTT butterflies. A serialized modular unit shares its routing and multiplier bank across the 16-bit ML-KEM and 32-bit ML-DSA datapaths. A 200-MHz V80 evaluation covers all six standardized parameter sets against a matched, optimized cooperative software mapping. Stage, shared NTT, and arithmetic reuse accelerate complete single-request ML-KEM by $2.314\text{--}2.365\times$ and ML-DSA by $1.910\text{--}2.150\times$; eight-request batches gain $1.914\text{--}2.126\times$. A separate five-point bank sweep selects two multipliers: it removes 87.500% of NTT-hierarchy DSPs at a 2.963% NTTBF test-cycle and at most 0.143% request penalty in middle-set Pointer controls. The results show that GPU cryptographic hardware must be sized at request scope, not by primitive peak throughput.

Index Terms—post-quantum cryptography, ML-KEM, ML-DSA, Keccak, NTT, SIMT, RISC-V

I. INTRODUCTION

The standardization of ML-KEM and ML-DSA gives post-quantum key establishment and digital signatures a common deployment target [1], [2]. Although the two schemes provide different services, both repeatedly invoke SHA-3/SHAKE and polynomial arithmetic. Efficient implementations must also execute sampling, encoding, and, for signatures, input-dependent rejection loops. Accelerating an isolated transform or permutation is therefore insufficient: the useful outcome is lower latency or higher throughput for a complete cryptographic request, including the surrounding computation.

A programmable GPU offers two levels of parallelism for these workloads. Threads can cooperate within a polynomial or a hash state, while independent warps process different requests. GPU implementations of Kyber and Dilithium already exploit this structure through register reuse, kernel fusion, batching, and cooperative scheduling [3], [4]. ML-Cube further combines GPU execution and machine-learning accelerators, including a 25-thread Keccak mapping [5]. These results motivate a complementary question for an extensible SIMT processor: which remaining work merits specialized hardware once the software already exploits warp parallelism?

Our starting point is the cost shared by both schemes. In a historical single-lane RV32 software study, removing the Keccak permutation bodies reduces a complete ML-KEM-768

request by 65.65%, while independently removing NTT/INTT bodies reduces it by 12.67%. ML-DSA key generation shows the same ordering (Section II). These diagnostics identify two reusable targets; they do not predict the benefit over optimized GPU software. We therefore build a stronger comparison point: register-resident, warp-cooperative Keccak and NTT kernels, with parallel sampling, encoding, and auxiliary arithmetic enabled identically in every hardware comparison. Our reported gains measure specialization beyond that software mapping.

The obstacle is not a lack of arithmetic parallelism but the cost of making operands meet. Each NTT layer contains 128 independent butterflies. With coefficients distributed across lane registers, some partners lie in the same lane and others in lanes separated by an XOR distance. Keccak-f[1600] instead couples 25 64-bit words through column reductions, fixed permutations, and row-neighbor logic [6]. Generic shuffles express both patterns, but introduce instructions and dependencies at each exchange. Keeping the state in registers removes intermediate memory traffic without removing these communication costs. The two primitives thus favor different internal connections even when they share the same architectural register interface.

Prior hardware addresses parts of this problem at other boundaries. RISC-V software optimization [7] and finite-field instruction extensions [8], [9] reduce arithmetic cost; RISQ-V and Nannipieri et al. add coarser polynomial operations to scalar cores [10], [11]. Configurable lattice processors such as Sapphire and KaLi already reuse resources across protocols [12], [13], while KiD explicitly unifies Kyber/Dilithium NTT multiplication [14]. Full-protocol FPGA accelerators also exploit request batching [15]. Sharing arithmetic is therefore established; the issue here is how shared arithmetic consumes distributed SIMT operands and returns results through an existing GPU pipeline.

The hash interface presents a related choice. Prior work quantifies how state transfers and sponge input/output affect hardware hashing performance [16]. Recent RISC-V extensions retain hash state in private registers [17]; a published patent also describes a Keccak round executed collectively by a warp [18]. Building on these interfaces, we study the constraints of an RV32 SIMT core: handling 64-bit state with 32-bit destinations, preserving per-warp dependency tracking, and selecting the instruction granularity at complete-request scope. A faster permutation can have little effect once sampling, sponge I/O, or polynomial work limits execution.

We address three coupled design constraints. First, collective operations must fit ordinary issue and writeback: an

NTT butterfly logically has three inputs and two outputs, while a Keccak step mixes values from many lanes. Second, ML-KEM’s 16-bit and ML-DSA’s 32-bit coefficients require different reductions without duplicating the entire modular datapath. Third, provisioning all 16 pair products in parallel may waste resources when other instructions or warps hide their latency. Neither an isolated primitive benchmark nor a comparison against scalar C resolves these constraints.

We implement *communication-matched warp collectives* in Vortex [19]. The state remains in ordinary GPRs, and each instruction replaces a fixed communication step together with its arithmetic (Fig. 1). Two lanes jointly execute one NTT butterfly, compute one Montgomery product, and each receive one result. A shared serialized multiplier bank handles both moduli and also serves lane-local products. Keccak exposes either three stage operations or one fused round, with explicit low/high outputs on RV32. Every instruction returns a single architectural destination vector and retains no cryptographic state between instructions. We compare these register interfaces with a memory-coupled permutation engine and size the hardware using complete requests.

The resulting Stage-plus-NTT design, including modular-arithmetic reuse, accelerates complete single-request ML-KEM by $2.314\text{--}2.365\times$ and ML-DSA by $1.910\text{--}2.150\times$ over matched cooperative software on a 200-MHz Alveo V80. All six standardized parameter sets are covered. NTT specialization provides additional request-level gains after Keccak acceleration. A separate bank sweep finds that two multipliers replace 16 with at most 0.143% request-cycle increase in its measured workloads. Our contributions are:

- A stateless SIMT instruction interface for Keccak neighborhoods and XOR-paired NTT butterflies, with ordinary operand tracking and one destination per lane; controlled Stage, Round, and Pointer comparisons establish the importance of the execution boundary.
- A shared K/D modular unit that combines pair routing, modulus-specific reduction, and a serialized multiplier bank. It supports forward/inverse NTT and reuses the same arithmetic outside transforms without private polynomial storage.
- A complete-library, six-parameter FPGA evaluation with matched software baselines and separate Keccak, NTT, and arithmetic ablations. Concurrency and five-point multiplier sweeps connect primitive performance to request latency, batch throughput, and routed hardware cost.

II. BACKGROUND AND MOTIVATION

A. Shared hotspots and the scope of profiling

ML-KEM uses degree-256 polynomials modulo $q_K = 3329$; ML-DSA uses $q_D = 8380417$. Both use SHA-3/SHAKE for hashing and pseudorandom expansion, including polynomial sampling [1], [2], [6]. Table I reports our initial single-lane RV32 SimX diagnosis: one core, four warps of four threads, and no L2/L3. Each independent ablation removes one primitive’s body while preserving its call interface; the

TABLE I
INITIAL SOFTWARE COST DIAGNOSIS. INDEPENDENT ABLATION DELTAS, NOT ADDITIVE PHASE FRACTIONS OF THE CURRENT BOARD BASELINE.

Workload	Keccak	NTT/INTT
ML-KEM-768 complete request	65.65%	12.67%
ML-DSA-65 key generation (full RAM)	74.44%	7.62%

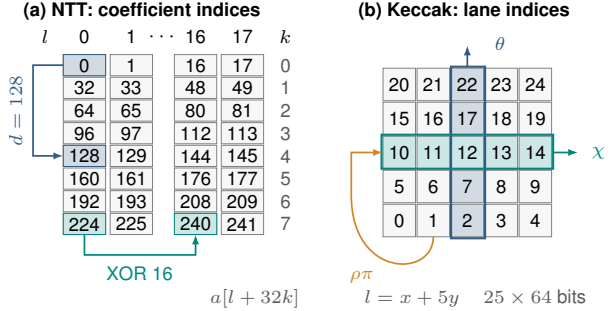


Fig. 1. Register placement determines the communication. (a) Selected NTT lanes: indices 0 and 128 pair within lane 0; 224 and 240 pair across lanes 0 and 16. (b) Keccak uses column parity (θ), a fixed permutation ($\rho\pi$, one edge shown), and row neighbors (χ).

reported fraction is $(C_0 - C_{-i})/C_0$. The instrumented baseline and its ablated variant share inputs and configuration. These are diagnostic runs with intentionally invalid outputs, not accelerated implementations.

KEM has 144 permutations and 24 transforms for the selected known-answer input. DSA is restricted to key generation: stubbing hashing changes signing retries and can prevent verification from terminating. The deltas include dependent scheduling and memory effects; they motivate specialization of both primitives, with benefit tested on correct complete requests.

B. Register placement defines the communication

With lane l holding $a[l + 32k]$, $0 \leq k < 8$, the first three forward NTT layers exchange registers within one lane. The remaining layers exchange lane l with $lxord$: $d = 16, 8, 4, 2$ for seven-layer ML-KEM, and additionally $d = 1$ for eight-layer ML-DSA. Coalesced loads and stores surround a transform whose intermediate state remains in GPRs.

Mapping Keccak word $A[x, y]$ to lane $x + 5y$ exposes column parity (θ), rotate-and-gather ($\rho\pi$), and row logic (χ). Each 64-bit word occupies two RV32 GPRs; rotations may need both halves even when an instruction returns one. Figure 1 contrasts these neighborhoods with NTT pairs. Prior polynomial accelerators organize computation around memory rows or accelerator-owned storage [20]–[23]; here the architectural lane vector is the boundary. Fixed communication can replace generic shuffles while leaving sampling and request control on the programmable core.

C. From software references to matched baselines

Portable C provides byte-exact reference outputs and the historical diagnosis. PQRV’s optimized scalar RISC-V Kec-

cak [7] provides a separate scalar comparison, not the denominator of the main board results. Our primary baseline, A, uses the register layouts above with software shuffles and the same surrounding warp optimizations as the hardware arms. B adds Keccak Stage; C adds shared NTT; D adds both; E additionally reuses modular hardware outside transforms. A–E isolate hardware gains under one software mapping. Separate matched configurations evaluate Keccak granularity and multiplier-bank size (Section IV).

III. DESIGN AND IMPLEMENTATION

A. Collective contract and core integration

Figure 2 places the units beside the integer ALU. An instruction consumes complete source vectors and the active mask, then returns one destination vector through the existing result arbiter. No operation contains a pointer, writes memory, or retains architectural state after writeback. Keccak requires a converged warp; NTT requires pair-closed masks. Padding lanes return zero. These constraints permit static wiring and allow different warps to interleave without saving a cryptographic context.

The decoder uses one custom opcode family and a compact function field for operation, low/high selection, NTT direction, modulus, and XOR distance. The scoreboard sees the same source and destination registers as the execution unit; no implicit register or memory dependence is introduced. A collective accepts an instruction only when its entire input vector and metadata can be captured. It holds warp ID, destination, mask, and lane results under output backpressure, so a stalled result cannot be paired with a later warp. This property is especially important for a serialized NTT instruction, which occupies several internal beats but commits exactly once.

B. Keccak: stages, rounds, and RV32 halves

For coordinates modulo five, the datapath implements

$$\begin{aligned} C[x] &= \bigoplus_y A[x, y], \\ T[x, y] &= A[x, y] \oplus C[x-1] \oplus \text{rol}(C[x+1], 1), \\ B[y, 2x+3y] &= \text{rol}(T[x, y], \rho[x, y]), \\ A'[x, y] &= B[x, y] \oplus (\neg B[x+1, y] \wedge B[x+2, y]). \end{aligned} \quad (1)$$

The *Stage* interface exposes θ , $\rho\pi$, and $\chi\iota$. Each has low/high RV32 forms returning the selected half. Six issues form a round. The *Round* interface fuses all stages and uses a literal round index. Its low/high instructions recompute the internal full word from identical operands, avoiding hidden state and multi-destination writeback. Two issues form an RV32 round. A memory-coupled engine that reads and writes the 200-byte state provides the coarser comparison point.

Stage has two pipeline registers and accepts one operation per cycle. Round registers parity, θ , $\rho\pi$, and $\chi\iota$ in four steps and also has initiation interval one. Its larger internal state explains why it can improve an isolated permutation while producing little request-level gain. For θ and $\rho\pi$, both half instructions read both old halves because rotation carries cross

the 32-bit boundary. Software therefore writes the first result to a temporary until the second issue has consumed the old pair. $\chi\iota$ reads one half and a warp-uniform round index. Round encodes the index as an immediate, enabling the 24 rounds to be expanded without a table load or branch in the permutation body.

C. NTT: one product, two lane results

For an XOR pair, pair-low lane p supplies a and twiddle ζ ; pair-high lane $h = p \text{ xor } d$ supplies b . A forward instruction computes

$$t = \text{Mont}(b\zeta), \quad (r_p, r_h) = (a + t, a - t). \quad (2)$$

One product thus serves both lanes, and each lane writes its own destination. The inverse instruction first forms a sum and difference, then multiplies the appropriate branch. Lane-local `NTTMUL.K/D` handles the first three layers, inverse scaling, ML-KEM base multiplication, and independent ML-DSA pointwise products.

A bank of M signed 32×32 multipliers serializes the 16 logical pair products without changing the ISA or binary. CT and ML-DSA inverse butterflies use $16/M$ beats; ML-KEM inverse and 32-lane scalar products use $32/M$. The direct latency and initiation interval are $6 + B$ and B , where B is the beat count. Operands are captured on the first handshake, lane results are assembled by index, and backpressure freezes the serializer and product pipeline. Pair routing, capture, and writeback are shared; only the constant-arithmetic reductions for q_K and q_D differ.

The shared bank preserves each library’s native residue representation rather than converting coefficients at its boundary. For ML-KEM, operands are sign-extended from 16 bits; a shift-add Montgomery factor and the constant 20159 Barrett product produce the forward and inverse results. ML-DSA uses the full signed 32×32 product, its Montgomery inverse, and q_D in the reduction stage. Both modes therefore use the same operand capture, multipliers, beat controller, and result assembly. Modulus selection changes only the constant paths after multiplication. This organization lets `NTTMUL.D` serve both butterfly products and ordinary pointwise products without a second arithmetic unit.

Twiddles are explicit lane operands; there is no twiddle ROM or internal layer counter. The paired encoding maps (a, b, ζ) and (r_p, r_h) onto two source vectors and one destination vector. Figure 3 shows M2 assigning 16 pairs to two multipliers over eight beats. Result assembly records lane indices and releases the vector after the final reduction. Other warps can execute on other functional units while the issuing warp waits on its destination dependency.

D. Complete-library integration

The native libraries retain their public APIs and byte-level outputs [24], [25]. Their Keccak and NTT hooks select the shared datapaths. Lane zero broadcasts primitive arguments, activates the cooperating lanes, and resumes scalar control when the primitive returns. One-shot SHAKE keeps its state

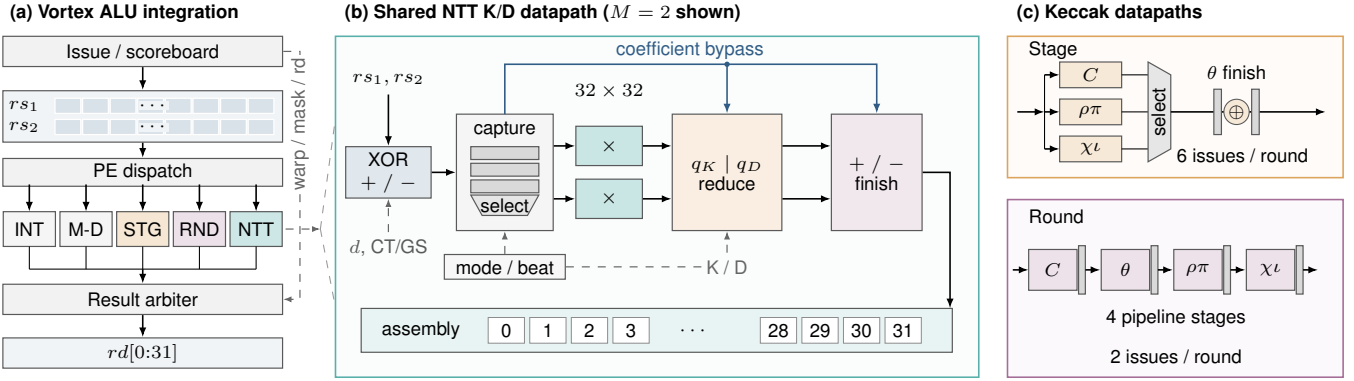
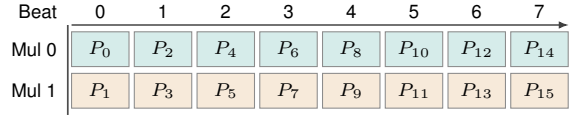


Fig. 2. Core integration and collective datapaths. (a) Dedicated PEs share operand dispatch and result arbitration with INT and multiply/divide. (b) Pair preparation includes GS pre-add/subtract. Both moduli share the multiplier bank, coefficient bypass, and result assembly. (c) Stage forms column parity C before its first register and completes θ after it; Round pipelines all four steps. Thin vertical bars denote pipeline registers.

(a) Two multipliers, sixteen pair products



(b) RV32 Round: preserve the old register pair

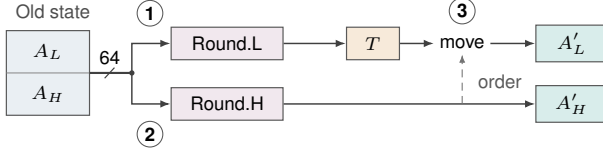


Fig. 3. Serialization and dependencies. (a) M2 assigns 16 pair products to two multipliers in eight CT or DSA-GS beats; pipeline latency is omitted. (b) Program order for an RV32 Round: both issues read the old halves, the low result remains in T , and the final move updates A_L .

TABLE II
COLLECTIVE INTERFACES ON RV32. ISSUE COUNTS ARE PER KECCAK ROUND OR PER CROSS-LANE NTT LAYER; B IS THE SERIALIZED BEAT COUNT.

Operation	Cooperation	Result/lane	Issues or II
$\theta, \rho\pi, \chi^l$	25 lanes	word half	6/round
Round	25 lanes	word half	2/round
NTTBF.K/D	XOR pair	coefficient	$II=B$
NTTMUL.K/D	lane-local	product	$II=B$

in registers across absorb, permutation, and squeeze; an incremental context is stored between calls. NTT kernels load eight coefficients per lane and perform all layers without intermediate memory. ML-DSA also reuses NTTMUL.D for ordinary pointwise products. The ML-DSA-65 five-term matrix-vector sum accumulates in 64-bit software and performs one final Montgomery reduction, preserving the library representation.

The leader controls allocation and rejection loops; warp kernels parallelize candidate filtering, encoding, noise generation, and signing arithmetic. All 32 lanes enter and leave each cooperative region together, using the converged or pair-closed masks required by the primitive. Intrinsic name every source

and destination for ordinary compiler dependency tracking.

Full-RAM ML-DSA reserves 88 KiB per request for sets 44/65 and 160 KiB for set 87. Leader-only allocation and reverse-order release reclaim temporaries after every signing attempt. All six parameter sets compare keys, ciphertexts or signatures, and shared secrets byte for byte with portable C.

E. Model and RTL verification

The SimX model mirrors RTL bank beats, pipeline latency, and result stalls. Processor tests exercise decode, scoreboard hazards, and writeback; XRT adds the AFU integration used by complete applications. Unit tests cover all five XOR distances, both directions and moduli, masks, backpressure, and consecutive warp metadata. Lane results are checked against scalar references before transform boundary vectors and complete-library known-answer tests. Matched SimX/XRT binaries must retire identical instruction counts with cycle gaps below a fixed 5% bound. All 60 single/eight-request A–E cases across the six sets pass both models with identical instruction counts and at most 4.422% cycle difference. Their five board repetitions also match the XRT instruction counts. Independent post-route cores measure each hardware choice.

IV. EVALUATION

A. Experimental Methodology

Table III specifies the V80 configuration: the NTT bank has two multipliers, the D-cache is write-through, and L2/L3 are disabled. All physical implementations use Vivado 2025.1 OPT3; the board clock reads back at 200 MHz before and after measurement. A request includes keypair/encapsulation/decapsulation for KEM or keypair/sign/verify for DSA. Device makespan excludes host transfers; we report the median of five runs after one warmup.

A–E retain the same warp sampling, KEM encoding/noise/linear arithmetic, and DSA SHAKE extraction, accumulation, and signing arithmetic. Zeroization uses the warp with one worker and the leader otherwise. Opcode audits verify no PQC instructions in A and no NTT instructions in A/B, including Montgomery conversion. Modes share the

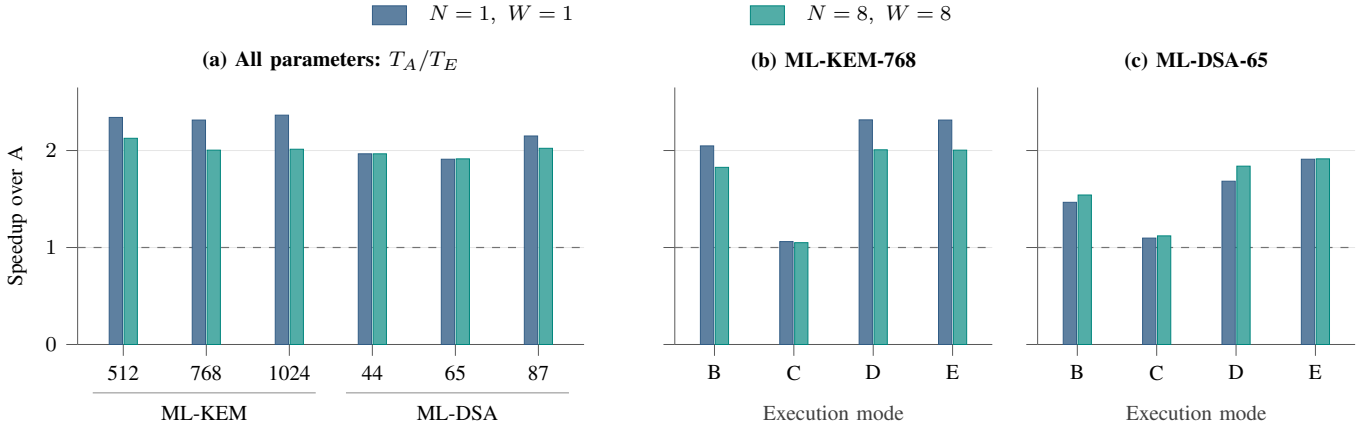


Fig. 4. 200-MHz V80 complete-request speedups over matched cooperative software (A). B enables Keccak Stage; C enables shared NTT; D enables both; E also uses modular products outside transforms. Blue/teal denote one/eight requests, with one worker per request. (a) A/E across all six sets; (b,c) B–E ablations for KEM-768 and DSA-65. Dashed lines mark software A at $1\times$. Every arm uses the same software mapping; ratios divide five-run median makespans.

TABLE III
PRIMARY EXPERIMENTAL CONFIGURATION.

Processor	RV32IM, one core, F/D disabled
Warp organization	8 resident warps, 32 lanes
On-core memory	16 KiB I\$, D\$, and local memory
Workloads	ML-KEM-512/768/1024; ML-DSA-44/65/87
Timing	XRT RTL integration; SimX controls; V80 silicon
Physical flow	Alveo V80, 200 MHz, Vivado 2025.1

ordered inputs and board image. All 156 board cells (780 timed runs) pass byte-exact checks; XRT validates the 60 A–E single/eight-request cases. Separate fusion and bank controls retain their archived write-back configurations, identified below.

B. Complete-Request Acceleration

Figure 4 uses a single complete-mapping cohort. At one request, E accelerates ML-KEM by $2.314\text{--}2.365\times$ and ML-DSA by $1.910\text{--}2.150\times$ over A. At eight requests, the ranges are $2.004\text{--}2.126\times$ and $1.914\text{--}2.023\times$, respectively. Accelerated single-request latency is $5.912\text{--}14.757$ ms for KEM and $51.264\text{--}107.181$ ms for DSA.

C. Contributions of the Hardware Paths

The ablation separates each hardware contribution. Across parameter sets and one/eight requests, Stage alone gives $1.827\text{--}2.124\times$ for KEM and $1.466\text{--}1.744\times$ for DSA (A/B). With Stage fixed, shared NTT adds $1.082\text{--}1.153\times$ for KEM and $1.118\text{--}1.194\times$ for DSA (B/D). NTT therefore adds request-level value after Keccak acceleration.

Reusing the modular unit outside transforms adds $1.040\text{--}1.146\times$ for DSA (D/E), with no new RTL. KEM’s corresponding range is only $0.998\text{--}1.024\times$; set 768 is within 0.2% of D. All headline results use E. Within a matched case, $(A/B)(B/D)(D/E) = A/E$; ratios from separate cohorts are not combined.

D. Granularity and Arithmetic Capacity

Figure 5(a) compares Round with equally unrolled Stage. At one and eight active warps, fusion improves dependent permutation throughput by $3.168\times$ and $3.291\times$, respectively. Separate SimX complete-KEM launch-cycle gains are only $1.028\times$ and $1.033\times$; the eight-request fusion control has no paired XRT run. A separate instrumented XRT control attributes only 4.964% of request time to Stage permutation; sponge I/O, sampling, and other work limit further gains. This profile does not describe the six-parameter board cohort. Fixed-mapping XRT controls also find Round $1.130\times$ faster than Pointer at one request, but Pointer $1.374\times$ faster at eight. Fusion and offload therefore require request-level evaluation at the intended concurrency.

Figures 5(b,c) distinguish test-program cycles from complete requests. The former uses historical RV32IMF W4T32 processor RTL, including loads, stores, and loops. The latter fixes Pointer Keccak in RV32IM W8T32 write-back XRT runs for KEM-768/DSA-65 at one/eight requests. Each timing experiment preserves its binary and inputs across banks.

M16 minimizes NTTBF test cycles. M8 and M4 increase them by 0.181% and 0.715%; M2 increases them by 2.963%, with at most 0.143% request overhead. The independent routed study in Fig. 5(d) finds that M2 removes 71.074% of NTT LUTs, 46.131% of FFs, and 87.500% of DSPs. M1 saves only five more core DSPs and increases NTT FFs, while test cycles rise by 25.043% over M16. M2 retains most resource savings with little measured request penalty.

All five NTTBF tests retire 59,534 instructions, pass 1,044 vectors, and show at most 0.959% SimX/RTL difference. Request sensitivity is much smaller than test-program sensitivity; identical instruction counts exclude a software change but do not isolate the contribution of latency hiding.

E. Concurrency and Batch Size

Figure 6 separates occupancy from batch size. On eight fixed inputs, increasing workers from one to eight raises

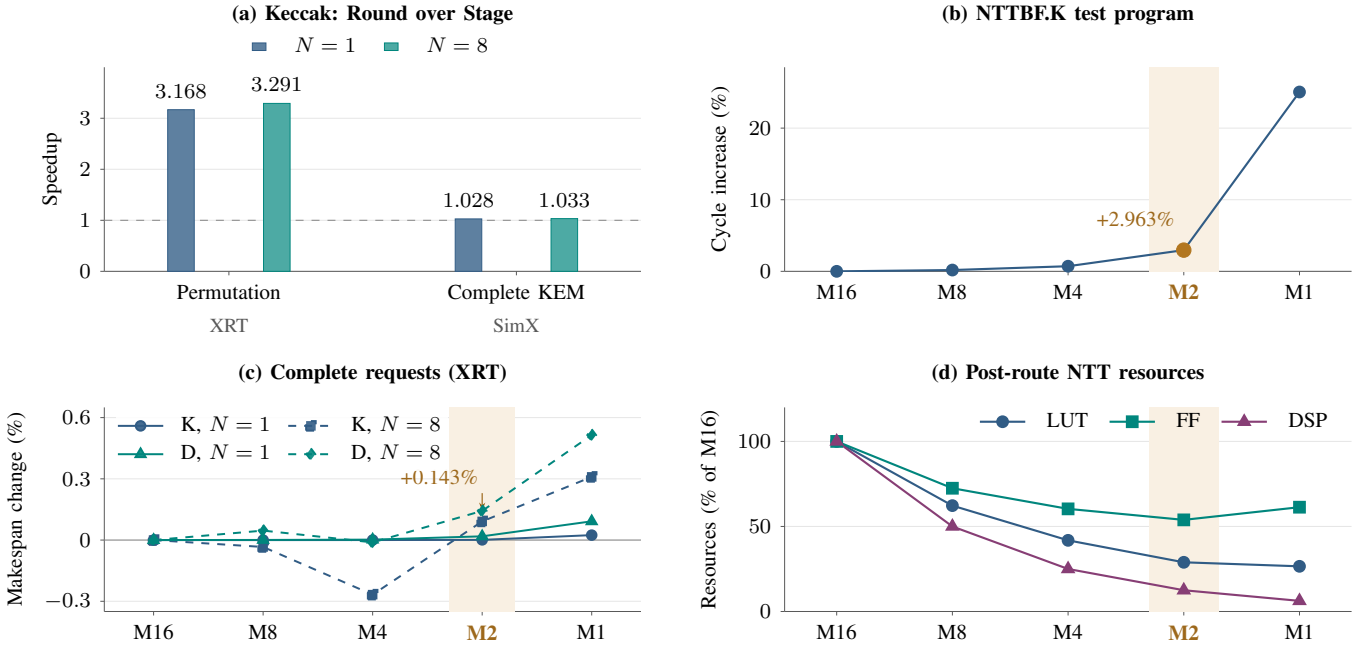


Fig. 5. Granularity and capacity in separate controlled studies. (a) Round/Stage: XRT permutation intervals and SimX KEM launch cycles, with $N = 1/8$ active warps. (b) Historical RV32IMF W4T32 NTTBF.K test-program cycles. (c) RV32IM W8T32 write-back XRT requests with Pointer Keccak: K=ML-KEM-768, D=ML-DSA-65. (d) Independent 200-MHz write-through NTT resources. Bank comparisons use M16 as reference; gold bands mark M2.

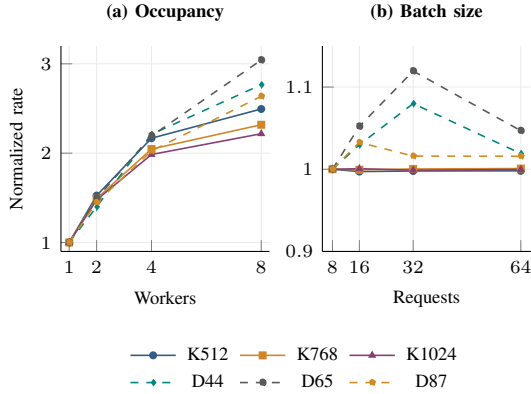


Fig. 6. 200-MHz board throughput for E. (a) Eight fixed inputs, normalized to one worker. (b) Eight workers, normalized to an eight-request batch. Curves cover all six parameter sets.

E throughput by $2.218\text{--}3.042\times$. With eight workers fixed, extending batches from eight to 64 changes throughput to only $0.998\text{--}1.047\times$ its eight-request value. Intermediate DSA batches peak at $1.119\times$, so throughput is not monotonic. Batches use synchronized waves. DSA signing retries vary by input; extending a batch also extends its input prefix, so its curve includes both amortization and workload variation.

F. Routed Cost and Design Scope

All cores in Table IV close without routing errors. M2 reduces whole-core LUTs/DSPs by $4.190\%/39.773\%$ over M16; Stage adds 1.289% LUTs to M2. At M16, Round uses

TABLE IV
INDEPENDENT RV32IM POST-ROUTE CORES AT 200 MHz. M16 VARIES
KECCAK GRANULARITY; M2 IS MEASURED BOTH ALONE AND WITH
STAGE.

Variant	LUT	FF	DSP	WNS (ns)
M16 base	233,759	136,451	176	+0.078
M16 + Stage	237,073	138,503	176	+0.116
M16 + Round	236,625	142,541	176	+0.052
M16 + Pointer	243,399	139,407	176	+0.080
M2 base	223,964	131,306	106	+0.048
M2 + Stage	226,852	133,368	106	+0.145

fewer LUTs but more FFs than Stage, so neither dominates in resource cost. The board AFU also includes optional engines: 243,073 LUTs, 181,881 FFs, 106 DSPs, and 0.001 ns setup slack. Board ratios compare software paths within this image.

Six-parameter results validate Stage+M2, without repeating the bank sweep or establishing joint optimality. Other transform intensity or concurrency may favor another design point. Energy and host-to-host latency are unmeasured.

V. CONCLUSION

Communication-matched collectives expose Keccak neighborhoods and NTT pairs through ordinary lane registers, with shared K/D modular arithmetic. All six parameter sets gain $1.910\text{--}2.365\times$ for complete single requests over cooperative software at 200 MHz. Independent granularity and bank studies connect primitive timing to request benefit and routed cost.

REFERENCES

- [1] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST, Tech. Rep. FIPS 203, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [2] —, “Module-lattice-based digital signature standard,” NIST, Tech. Rep. FIPS 204, 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [3] X. Ji, J. Dong, T. Deng, P. Zhang, J. Hua, and F. Xiao, “HI-Kyber: A novel high-performance implementation scheme of Kyber based on GPU,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 6, pp. 877–891, 2024.
- [4] S. Shen, H. Yang, W. Dai, H. Zhang, Z. Liu, and Y. Zhao, “High-throughput GPU implementation of Dilithium post-quantum digital signature,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 11, pp. 1964–1976, 2024.
- [5] T. Zhou, F. Zheng, Z. Xie, W. Tang, G. Fan, Y. Ning, Y. Bian, J. Lin, and J. Jing, “ML-Cube: Accelerating module-lattice-based cryptography using machine learning accelerators with a memory-less design,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2025, pp. 1829–1843.
- [6] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” NIST, Tech. Rep. FIPS 202, 2015. [Online]. Available: <https://csrc.nist.gov/pubs/fips/202/final>
- [7] J. Zhang, Y. Yan, J. Huang, and Ç. K. Koç, “Optimized software implementation of Keccak, Kyber, and Dilithium on RV{32,64}IM{B}{V},” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 632–655, Dec. 2024.
- [8] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri, “ISA extensions for finite field arithmetic: Accelerating Kyber and NewHope on RISC-V,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 3, pp. 219–242, 2020.
- [9] R. Bevin, A. Khalid, M. Imran, and M. O’Neill, “Accelerating CRYSTALS-Kyber and Dilithium via a single Montgomery reduction ISE on RISC-V,” in *Proc. IEEE 38th Int. System-on-Chip Conf. (SOCC)*, 2025, pp. 1–6.
- [10] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: Tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 4, pp. 239–280, 2020.
- [11] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150 798–150 808, 2021.
- [12] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, “Sapphire: A configurable crypto-processor for post-quantum lattice-based protocols,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 4, pp. 17–61, 2019.
- [13] A. Aikata, A. C. Mert, M. Imran, S. Pagliarini, and S. S. Roy, “KaLi: A crystal for post-quantum security using Kyber and Dilithium,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 70, no. 2, pp. 747–758, 2023.
- [14] S. Mandal and D. B. Roy, “KiD: A hardware design framework targeting unified NTT multiplication for CRYSTALS-Kyber and CRYSTALS-Dilithium on FPGA,” in *Proc. 37th Int. Conf. VLSI Design and 23rd Int. Conf. Embedded Systems (VLSID)*, 2024, pp. 455–460.
- [15] X. Carril, C. Kardaris, J. Ribes-González, O. Farràs, C. Hernandez, V. Kostalabros, J. U. González-Jiménez, and M. Moretó, “Hardware acceleration for high-volume operations of CRYSTALS-Kyber and CRYSTALS-Dilithium,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 17, no. 3, pp. 41:1–41:26, 2024.
- [16] P. Karl, J. Schupp, and G. Sigl, “Performance and communication cost of hardware accelerators for hashing in post-quantum cryptography,” *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5, pp. 66:1–66:31, 2025.
- [17] A. Dolmeta, V. Piscopo, M. Hutter, M. Martina, and G. Masera, “HORCRUX: A complete PQC RISC-V extension architecture,” arXiv:2607.13939, 2026, preprint.
- [18] M. B. Sullivan, S. Ghosh, D. R. Johnson, G. Suh, C. Joseph, and M. Y. Siu, “Warp-collective instructions to accelerate cryptographic primitives,” U.S. patent application publication US 2026/0222190 A1, Jul. 2026, applicant: NVIDIA Corp.; published July 30, 2026, Listing 3. [Online]. Available: <https://patents.justia.com/patent/20260222190>
- [19] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3d-graphics research,” in *Proc. 54th Annual IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021, pp. 754–766.
- [20] J. Park, S. Lee, and J. Lee, “NTT-PIM: Row-centric architecture and mapping for efficient number-theoretic transform on PIM,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [21] J. Zhang, M. Imani, and E. Sadredini, “BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [22] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W. jie Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng, H. Liu, T. Chu, C. Hong, C. Wei, D. Niu, and Y. Xie, “CHAM: A customized homomorphic encryption accelerator for fast matrix-vector product,” in *Proc. Design Automation Conf. (DAC)*, 2023, pp. 1–6.
- [23] A. Putra, C. M. Duong, A. Q. Pham, and J.-Y. Kim, “ZK-Flex: A flexible and scalable framework for accelerating zero-knowledge proofs,” in *Proc. Design Automation Conf. (DAC)*, 2026.
- [24] PQ Code Package contributors, “mlkem-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mlkem-native>
- [25] —, “mldsa-native,” Software repository, 2026, local dependency revision is recorded in the companion artifact. [Online]. Available: <https://github.com/pq-code-package/mldsa-native>